CSE 6220/CX 4220
Introduction to HPC

Lecture 7: Prefix Sums

Helen Xu
hxu615@gatech.edu

Georgia Tech College of Computing
School of Computational
Science and Engineering

Reminder: Exam 1 in class on Feb 12

- Efficiency / speedup / scalability analysis
- Analysis of multithreaded algorithms (work / span)
- Cache complexity analysis / cache-oblivious analysis
- Prefix sums

Prefix Sum (Scan) Definition

Input: an array $x = [x_0, x_1, \ldots, x_{n-1}]$ of n elements

Output: an array $y = [y_0, y_1, \ldots, y_{n-1}]$ of running sums, where

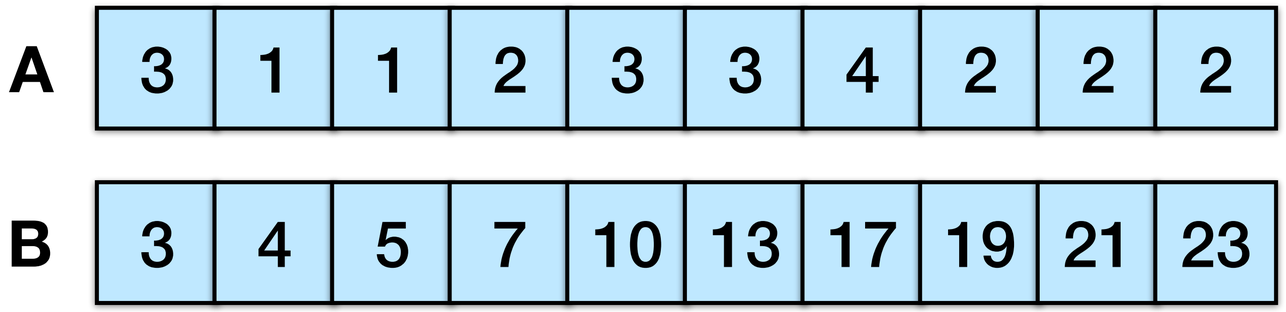
$$y_k = \begin{cases} x_0 & \text{if } k = 0 \\ x_k \oplus y_{k-1} & \text{if } k \geq 1. \end{cases}$$

Any binary associative operator (e.g., addition, multiplication, etc.)

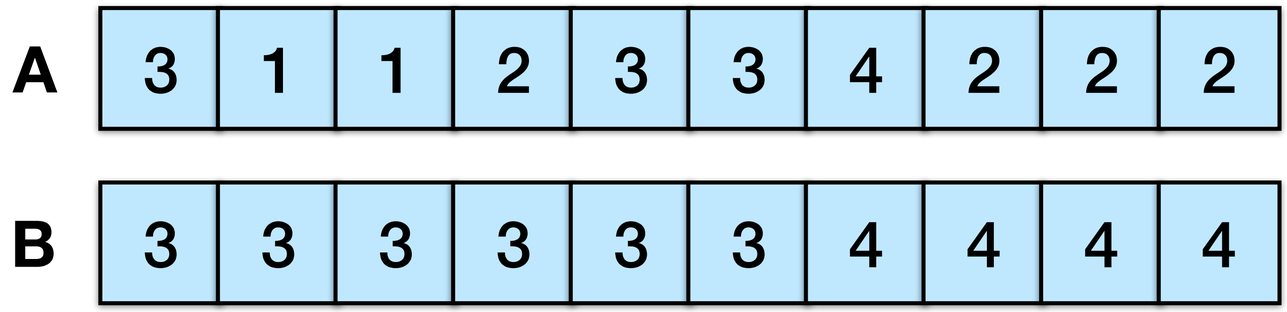
Scan Examples

Fill array with **partial reductions** from any binary associative operation

A = array
B = array
B = scan(A, +)



A = array
B = array
B = scan(A, max)



Inclusive and Exclusive Scans

Inclusive scan: includes x_k when computing y_k - as in our previous examples.

Another variant: **exclusive scan** does not include x_k when computing y_k.

A = array
B = array
B = inclusive_scan(A, +)

C = array
C = exclusive_scan(A, +)

A	3	1	1	2	3	3	4	2	2	2
B	3	4	5	7	10	13	17	19	21	23
C	0	3	4	5	7	10	13	17	19	21

Can easily get the inclusive version from the exclusive by adding the input element-wise.

For the other way, you need an inverse for the operator.

Suffix Scans

If prefix is “left to right”, suffix is just the reverse “right to left”

Input: an array $x = [x_0, x_1, \ldots, x_{n-1}]$ of n elements

Output: an array $y = [y_0, y_1, \ldots, y_{n-1}]$ of running sums, where

$$y_k = \begin{cases} x_{n-1} & \text{if } k = n-1 \\ x_k \oplus y_{k+1} & \text{if } k < n-1. \end{cases}$$

Any binary associative operator (e.g., addition, multiplication, etc.)

Shared-Memory Parallel Prefix

Can we parallelize a scan?

Serial scan takes n-1 operations.

The i-th iteration iteration of the loop **depends completely** on the (i-1)-th iteration.

```
y[0] = 0;
for (size_t i = 1; i < n; i++) {
  y[i] = y[i-1] + x[i];
}
```

From UC Berkeley CS267

First try: Parallel But Inefficient

Apply tree-like reduction at every element: put 1 processor at element 1, 2 at element 2, etc.

Span: $\lg(n)$

Work: $O(n^2)$

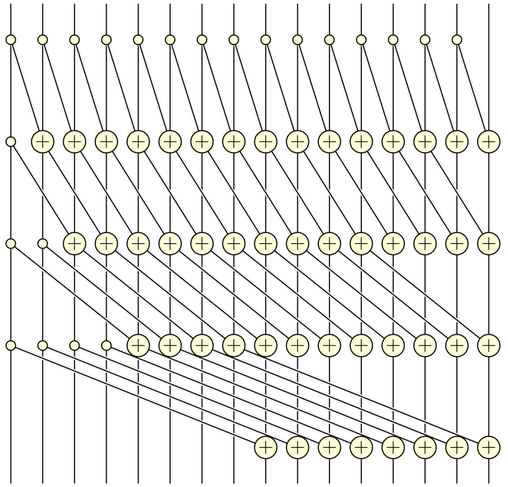
Work-efficient parallel algorithms perform **no more than a constant factor of work** over the best serial algorithm for the problem

A	1	2	3	4	5	6	7	8
B	1	3	6	10	15	21	28	36

Hillis-Steele Prefix Sum

for i = 0 up to $\log(n)$:
  parallel_for j = 0 up to n-1:
    if j < 2^i:
      $x_j^{i+1} \leftarrow x_j^i$
    else:
      $x_j^{i+1} \leftarrow x_j^i + x_{j-2^i}^i$

What is the work and span?



https://en.wikipedia.org/wiki/Prefix_sum

"Data-Parallel Algorithms," Hillis and Steele, CACM 1986.

Hillis-Steele Prefix Sum

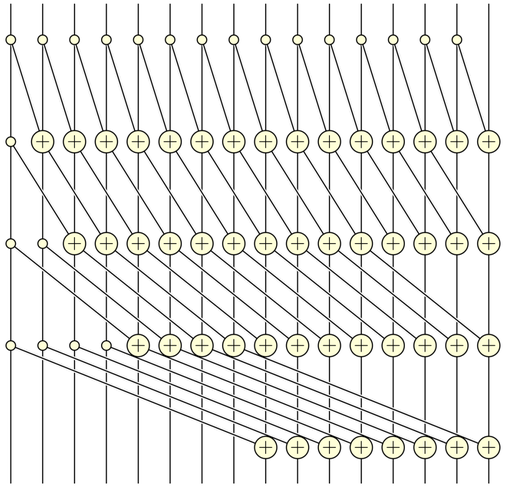
for i = 0 up to $\log(n)$:
  parallel_for j = 0 up to n-1:
    if j < 2^i:
      $x_j^{i+1} \leftarrow x_j^i$
    else:
      $x_j^{i+1} \leftarrow x_j^i + x_{j-2^i}^i$

What is the work and span?

Work = $O(n \lg n)$

Span = $O(\lg^2 n)$

Better, but still not work efficient



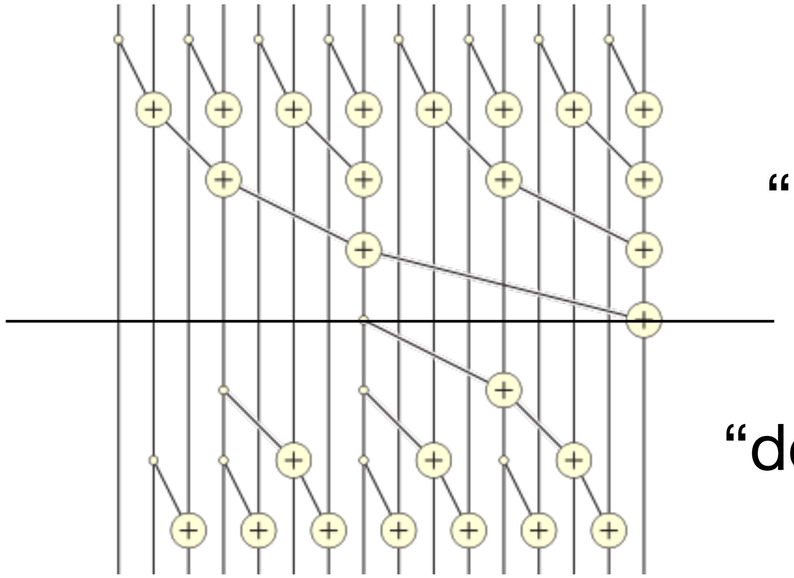
https://en.wikipedia.org/wiki/Prefix_sum

"Data-Parallel Algorithms," Hillis and Steele, CACM 1986.

Work-Efficient Parallel Prefix

Idea: Save the partial sums computed via parallel reduction (**upsweep**) and use those values in a **downsweep** pass to compute the total prefix.

The downsweep works by performing sums **down the prefix-sum tree**: at each step, each vertex at a given level passes its own value to its left child, and its right child gets the sum of the left child and the parent.



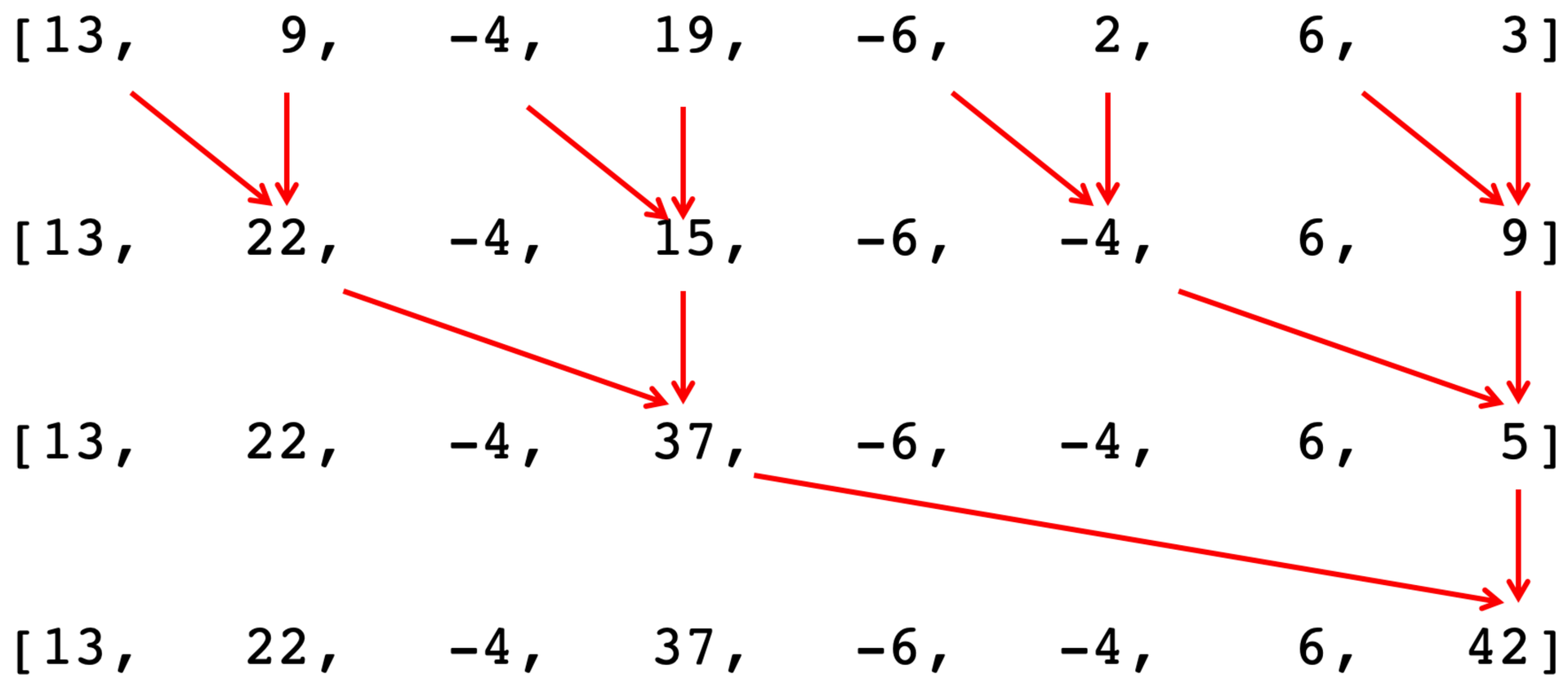
“upsweep”

“downsweep”

https://en.wikipedia.org/wiki/Prefix_sum

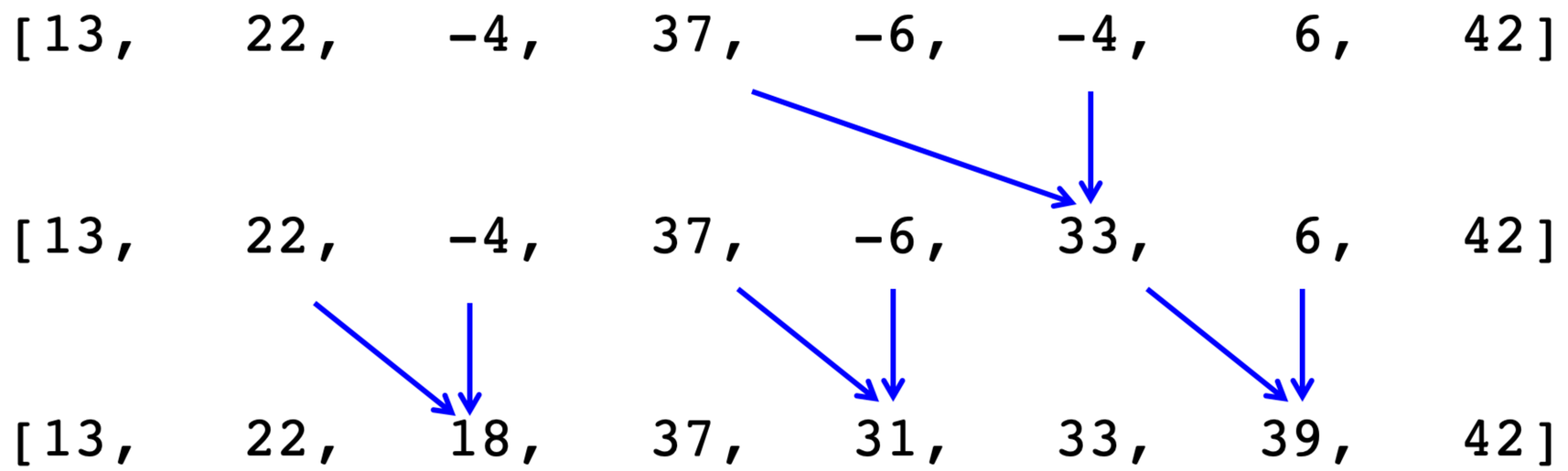
“Prefix Sums and Their Applications,” Blelloch, 1990.

Upsweep Example



Downsweep Example

For an inclusive scan, only the downsweep needs to change:



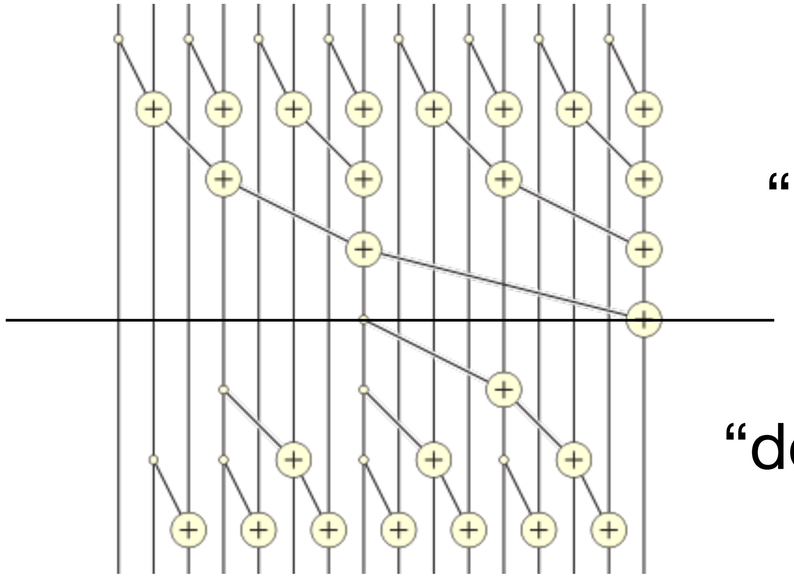
- Recall, we started with:

[13, 9, -4, 19, -6, 2, 6, 3]

Work-Efficient Parallel Prefix Pseudocode

Upsweep:
for d from 0 to $\lg(n) - 1$:
  parallel_for i from 0 to $n - 1$, i += 2^{d+1}:
    $A[i + 2^{d+1} - 1] \leftarrow A[i + 2^d - 1] + A[i + 2^{d+1} - 1]$

Downsweep:
for d from $\lg(n) - 1$ to 0:
  parallel_for i from $2^d - 1$ to $n - 1 - 2^d$, i += 2^{d+1}:
    if $i - 2^d \geq 0$:
      $A[i] = A[i] + A[i - 2^d]$



“upsweep”

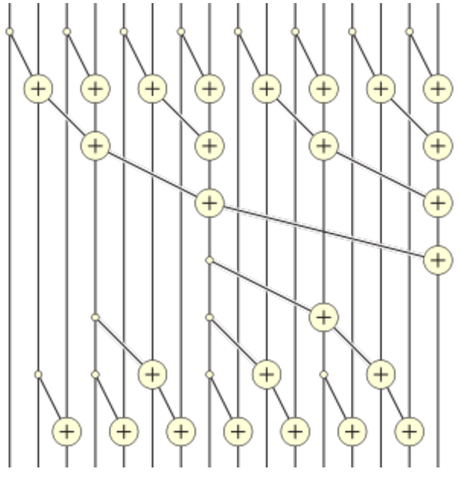
“downsweep”

https://en.wikipedia.org/wiki/Prefix_sum

“Prefix Sums and Their Applications,” Blelloch, 1990.

Analysis of Parallel Prefix

What is the work and span?



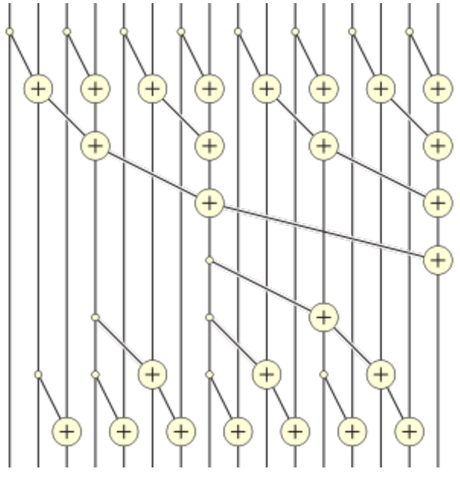
Analysis of Parallel Prefix

What is the work and span?

Work efficient

Work: $W(n) = W(n/2) + O(n) = O(n)$

Span: $S(n) = S(n/2) + O(1) = O(\lg n)$



Distributed-Memory Communication: Hypercubic Permutations

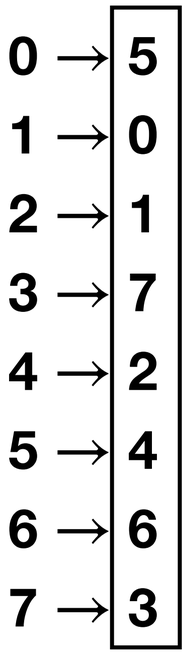
Modeling Communication

In a parallel communication step:

- Each processor can send at most 1 message
- Each processor can receive at most 1 message

Not necessary to send/receive from the same processor

Permutations can avoid conflicts of multiple processors sending to the same destination. For example, p = 8:

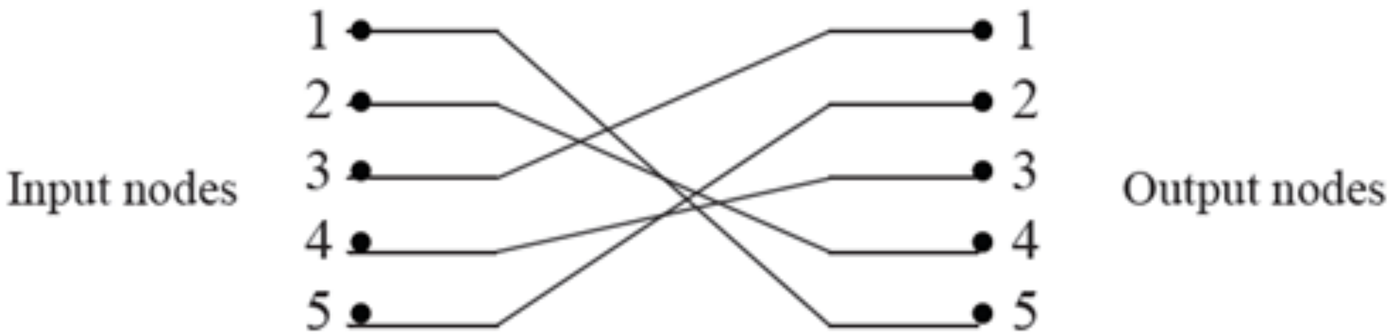


Can also write the permutation of dests with srcs in implicit order

Permutation Network Model

- Communication that corresponds to any permutation can be realized in parallel.
- p! communication patterns!

More variations, since not everyone has to participate, message sizes can different, and not everyone communicates exactly at the same time.



From Srinivas Aluru https://www.expertsmind.com/questions/concept-of-permutation-network-30138469.aspx

Idealized Networks

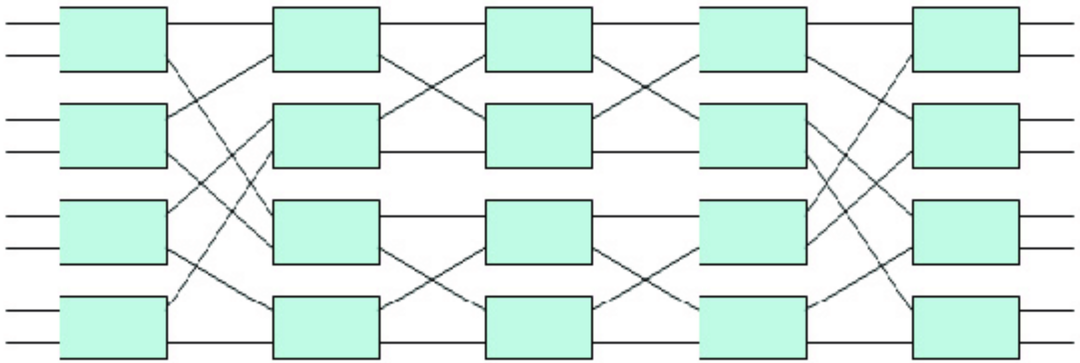
If you had $\Theta(p^2)$ links, an all-connected work could implement arbitrary permutations.

- Unfortunately, $\Theta(p^2)$ links is too many in practice.

The ideal number of links would be close to p.

- The Benes network can route permutations efficiently without conflicts with around $p \log p$ links.

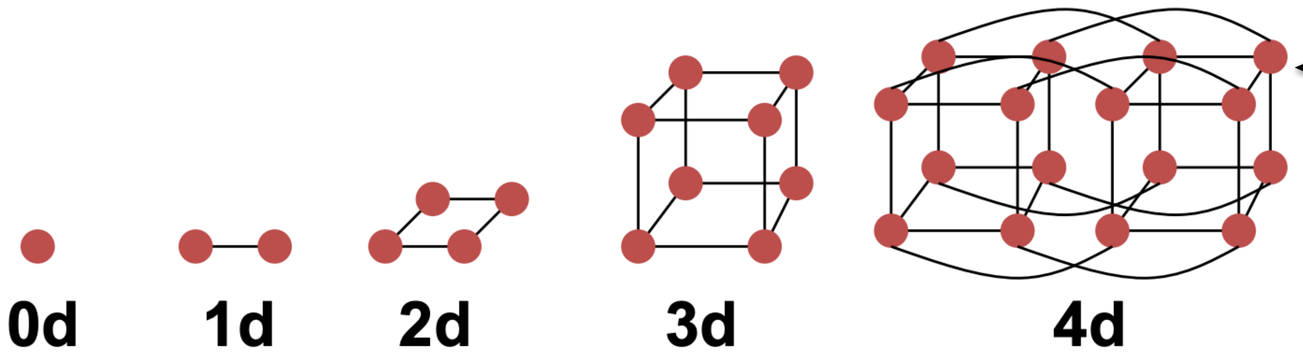
Impractical because it requires a centralized planner



From Srinivas Aluru https://www.researchgate.net/figure/A-Benes-network-88_fig9_269729211

Hypercubic Permutations

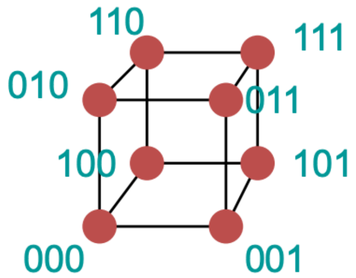
- **Number of nodes n = 2^d for dimension d.**
  - **Diameter = d.**
  - **Bisection bandwidth = n/2.**



Processors communicate only if their ranks differ in only one position

- **Popular in early machines (Intel iPSC, NCUBE).**
  - **Lots of clever algorithms.**

- **Greycode addressing:**
  - **Each node connected to d others with 1 bit different.**



Represent processor ranks with d-bit number binary numbers

Hypercubic Permutations

Fix some index j. Given processors with binary string representations, two processors that **differ in the j-th bit** communicate in hypercubic permutations.

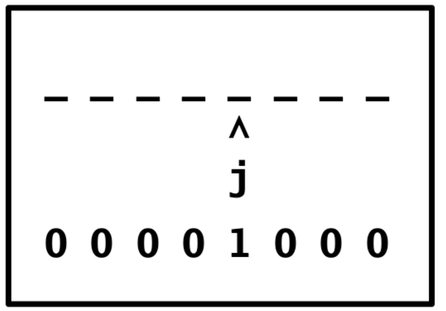
- Example $p = 8$, $d = 3$

j = 0	**j = 1**	**j = 2**
0=000 ↔ 001=1	0 ↔ 2	0 ↔ 4
2=010 ↔ 011=3	1 ↔ 3	1 ↔ 5
4=100 ↔ 101=5	4 ↔ 6	2 ↔ 6
6=110 ↔ 111=7	5 ↔ 7	3 ↔ 7

Toy Problem Using Hypercubic Permutations

- Given n numbers, compute their sum using p processors.
- Serial runtime: $T(n,1) = \Theta(n)$
- Parallel runtime: $T(n,p) = \Theta((n/p) + \lg p)$

Finding the Sum of n Numbers

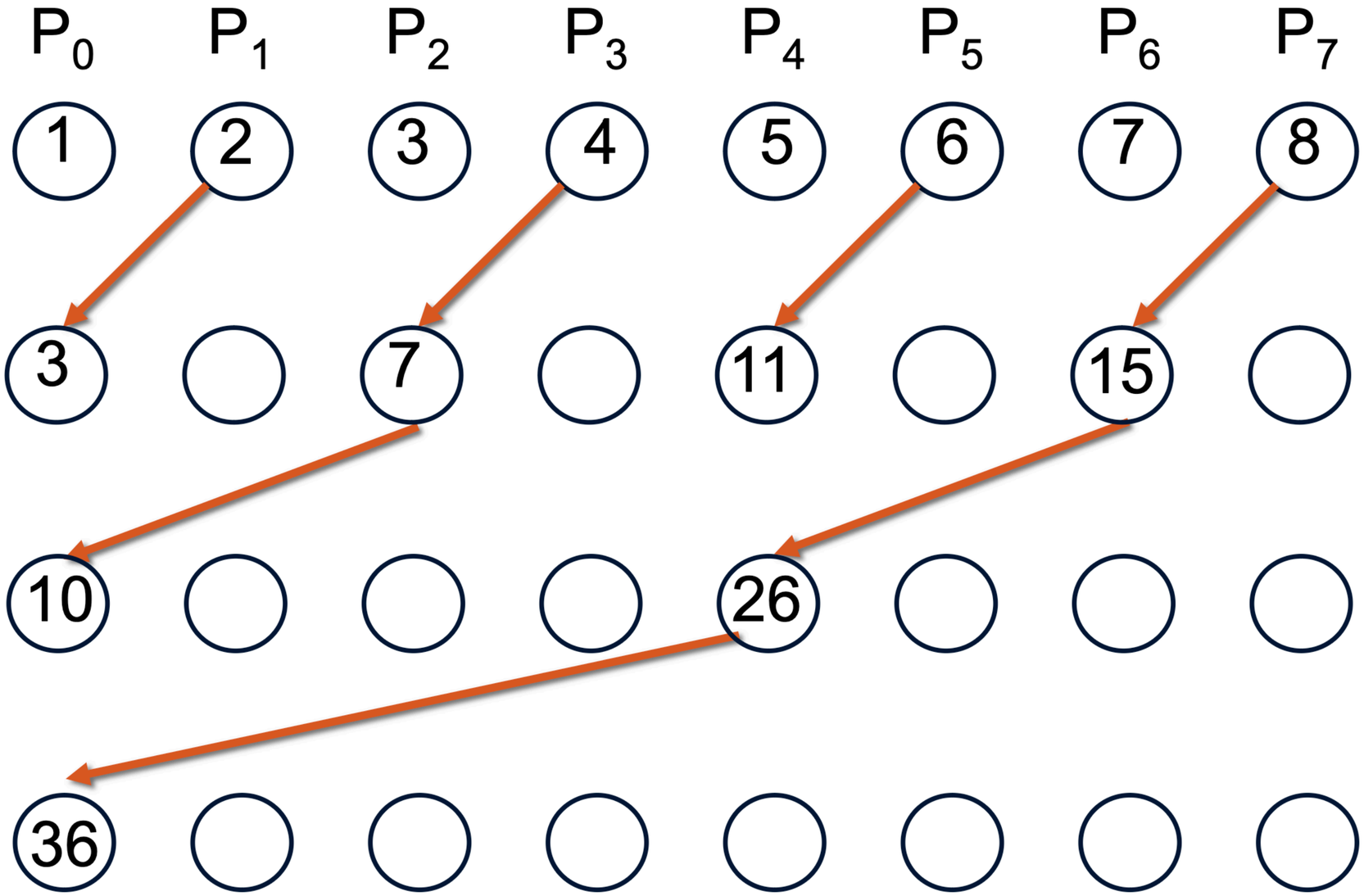


AND, XOR : bitwise operators

```
Algorithm (for P_i)
sum ← add local n/p numbers
for j=0 to d-1 do

     if ((rank AND 2^j)≠ 0)
          send sum to (rank XOR 2^j)
     else
          receive sum' from (rank XOR 2^j)
          sum = sum + sum'
endfor
if (rank=0)
     print sum
```

Finding the Sum of n Numbers



From Srinivas Aluru

Finding the Sum of n Numbers

Algorithm (for P_i)

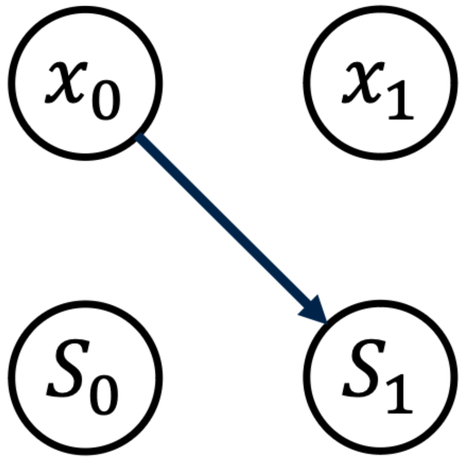
```
sum ← add local n/p numbers
for j=0 to d-1 do

     if ((rank AND 2^j) ≠ 0)
          { send sum to (rank XOR 2^j); exit; }
     else
          receive sum' from (rank XOR 2^j)
          sum = sum + sum'
endfor
if (rank=0)
     print sum
```

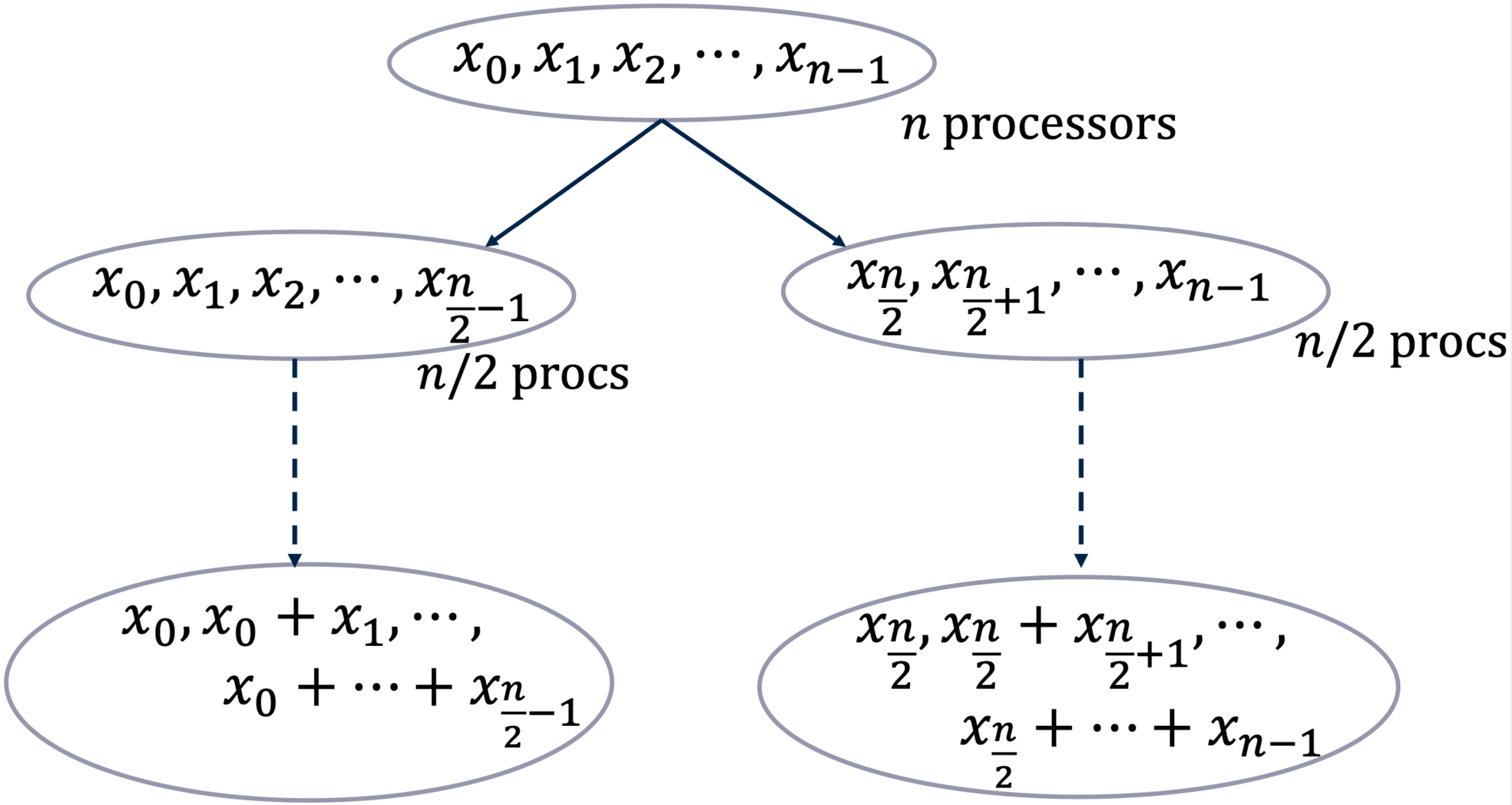
Distributed-Memory Parallel Prefix

Divide-and-conquer parallel prefix

Base case of two elements:



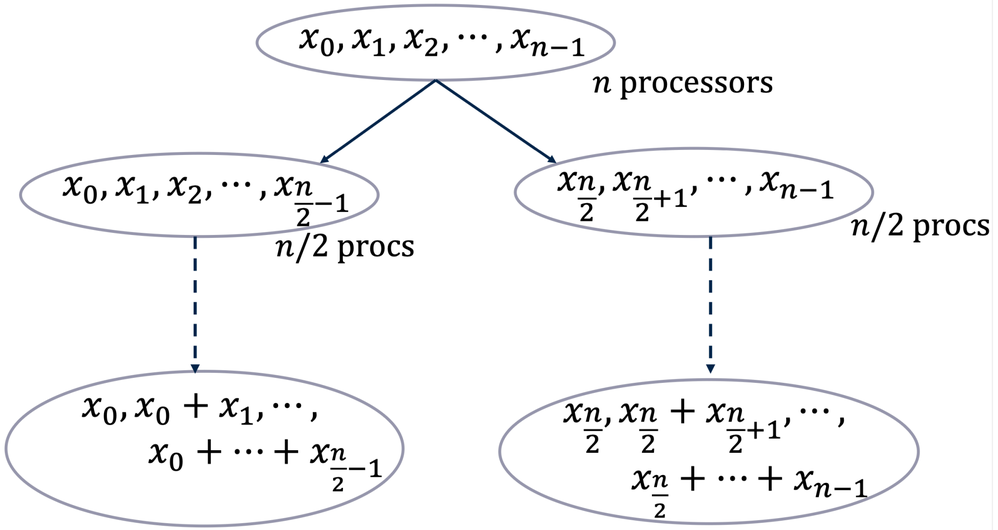
Divide-and-conquer parallel prefix



Analysis of Initial Divide-and-Conquer Attempt

- To merge the two recursive calls into the final correct answer, $S_{n/2-1}$ needs to be communicated to all processors on the right.
- $T(n,n) = T(n/2,n/2) + \Theta(\lg n) = \Theta(\lg^2 n)$

To broadcast rightmost sum in the left half to all elements in the right half



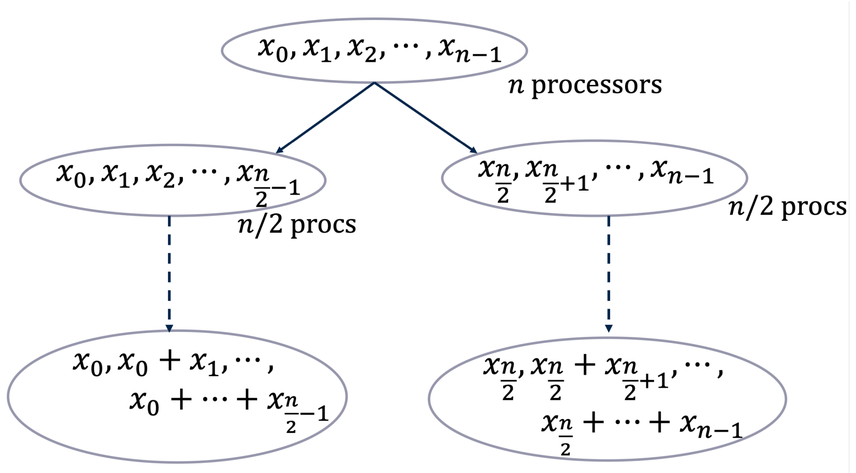
Can we do better? i.e., $T(n,n) = \Theta(\lg n)$?

A Cute Trick: Computing More for Better Efficiency

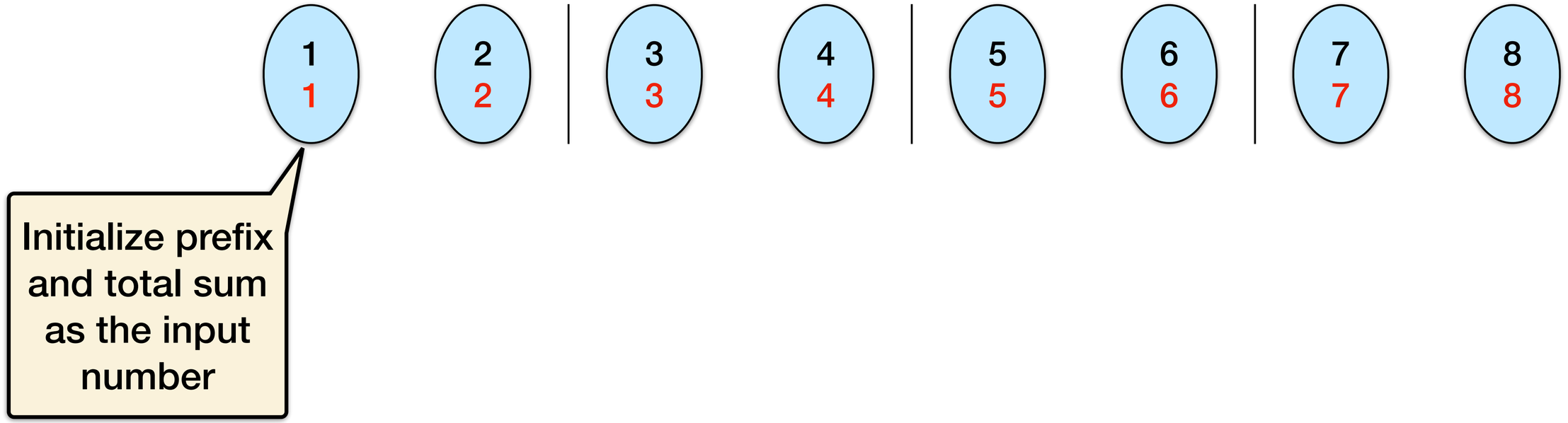
Problem: only one processor on the left side knows the total sum of that half. What if **every processor on the left half knew the total sum**?

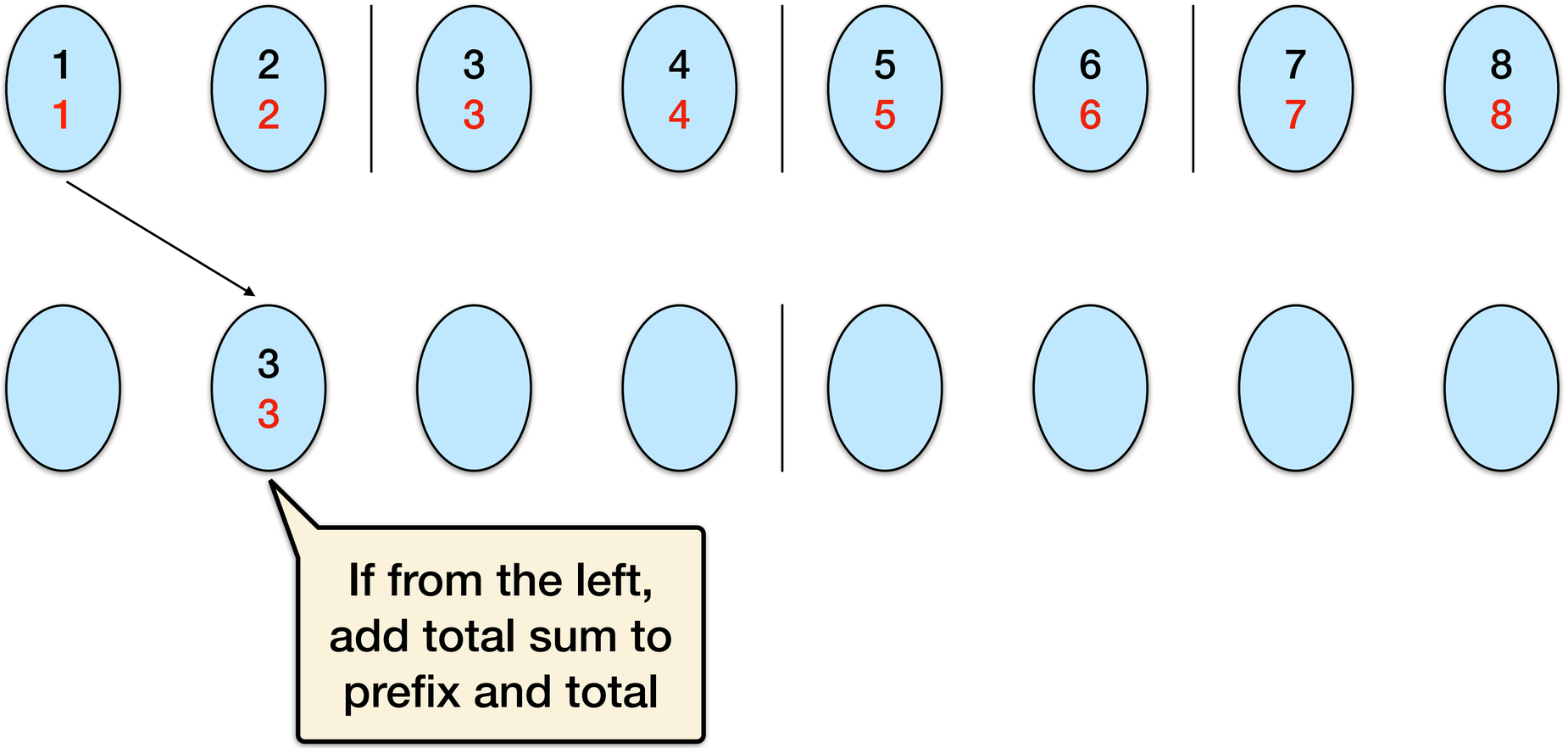
- Can then pair left and right according to a hypercubic permutation (flip the top bit)

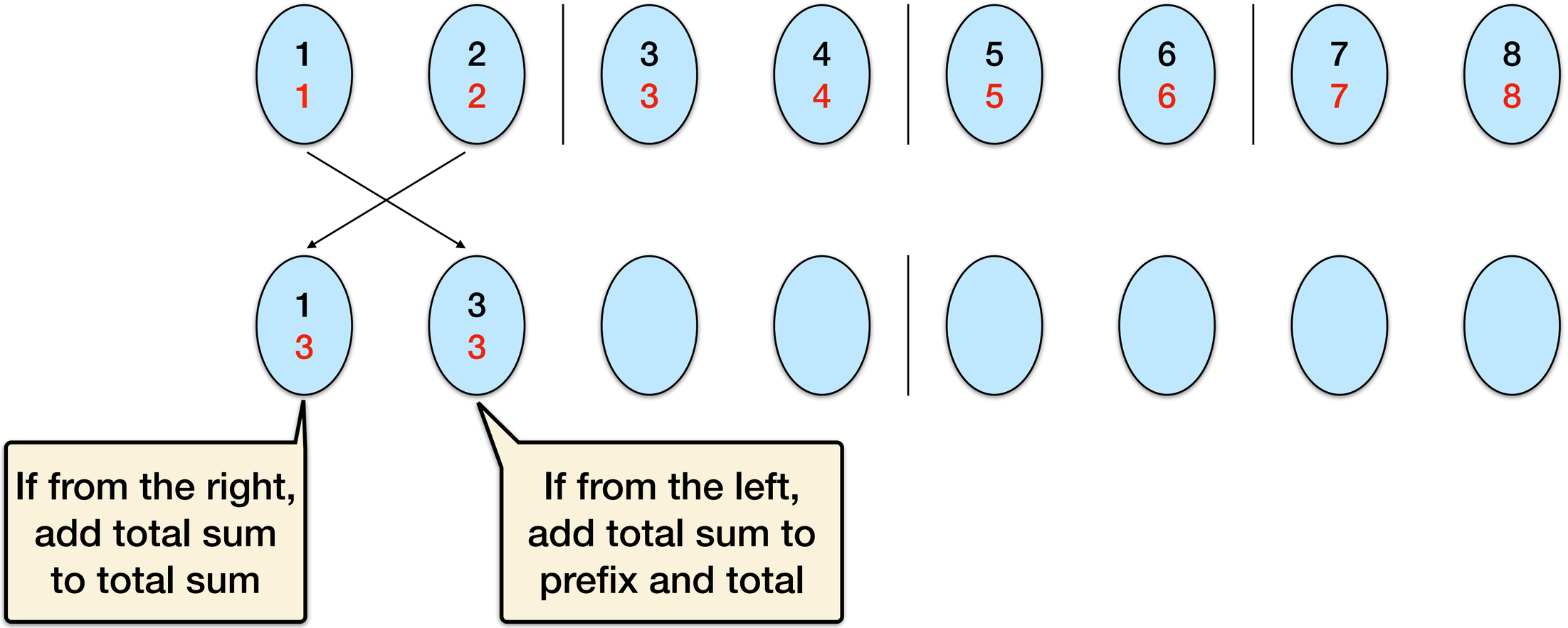
Insight: Compute **the total sum** on every processor on the left side in addition to the prefix sum.

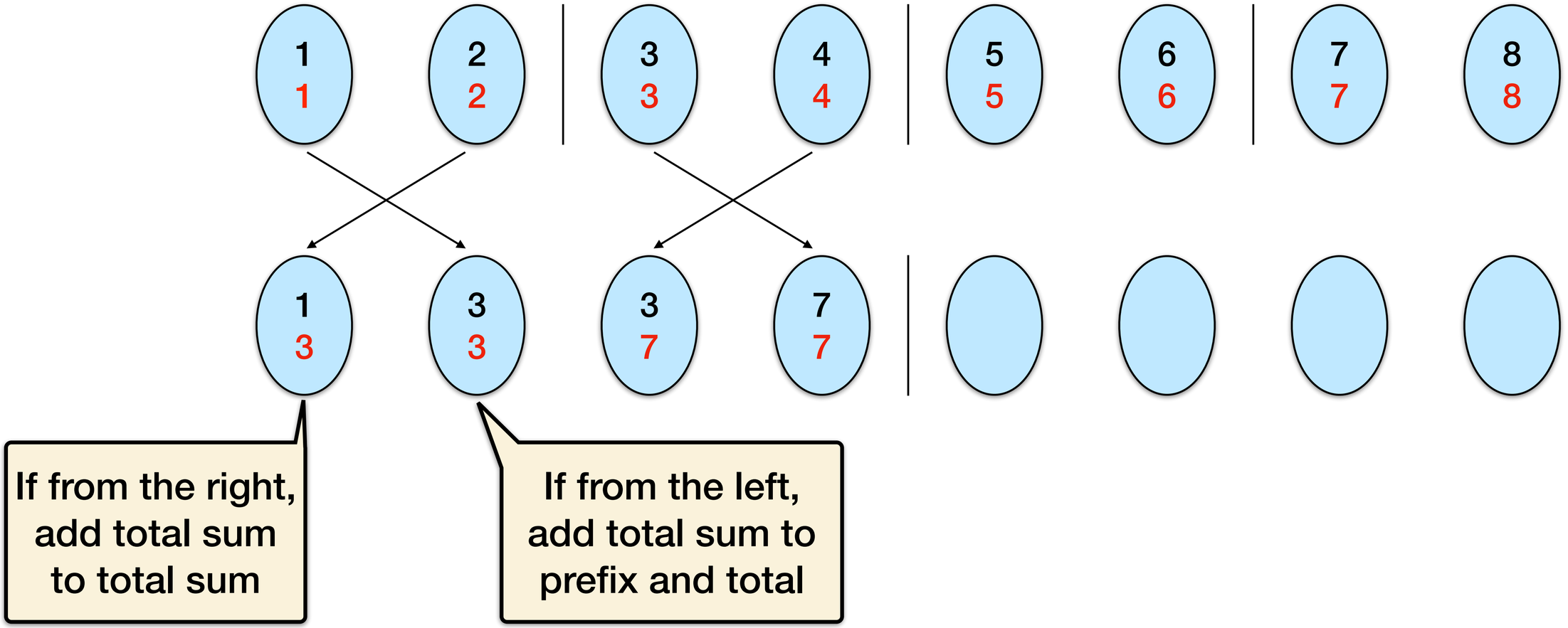


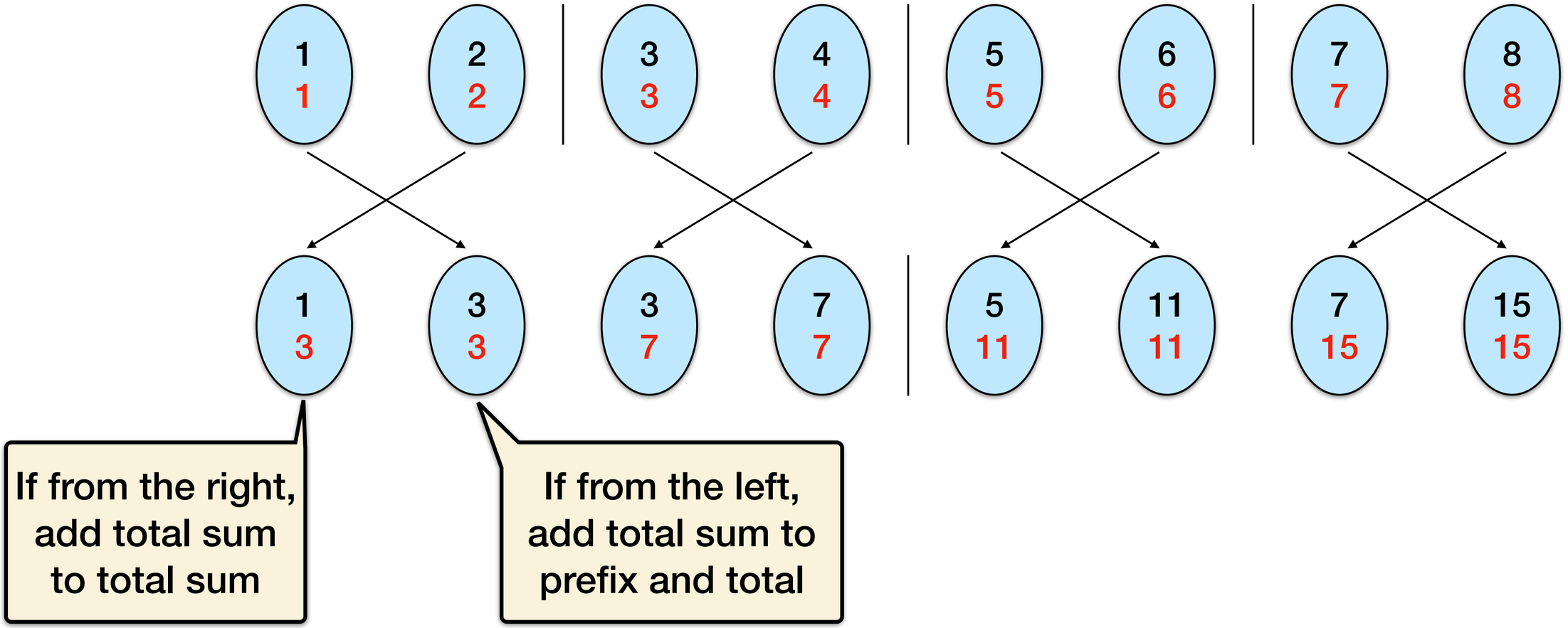
From Srinivas Aluru

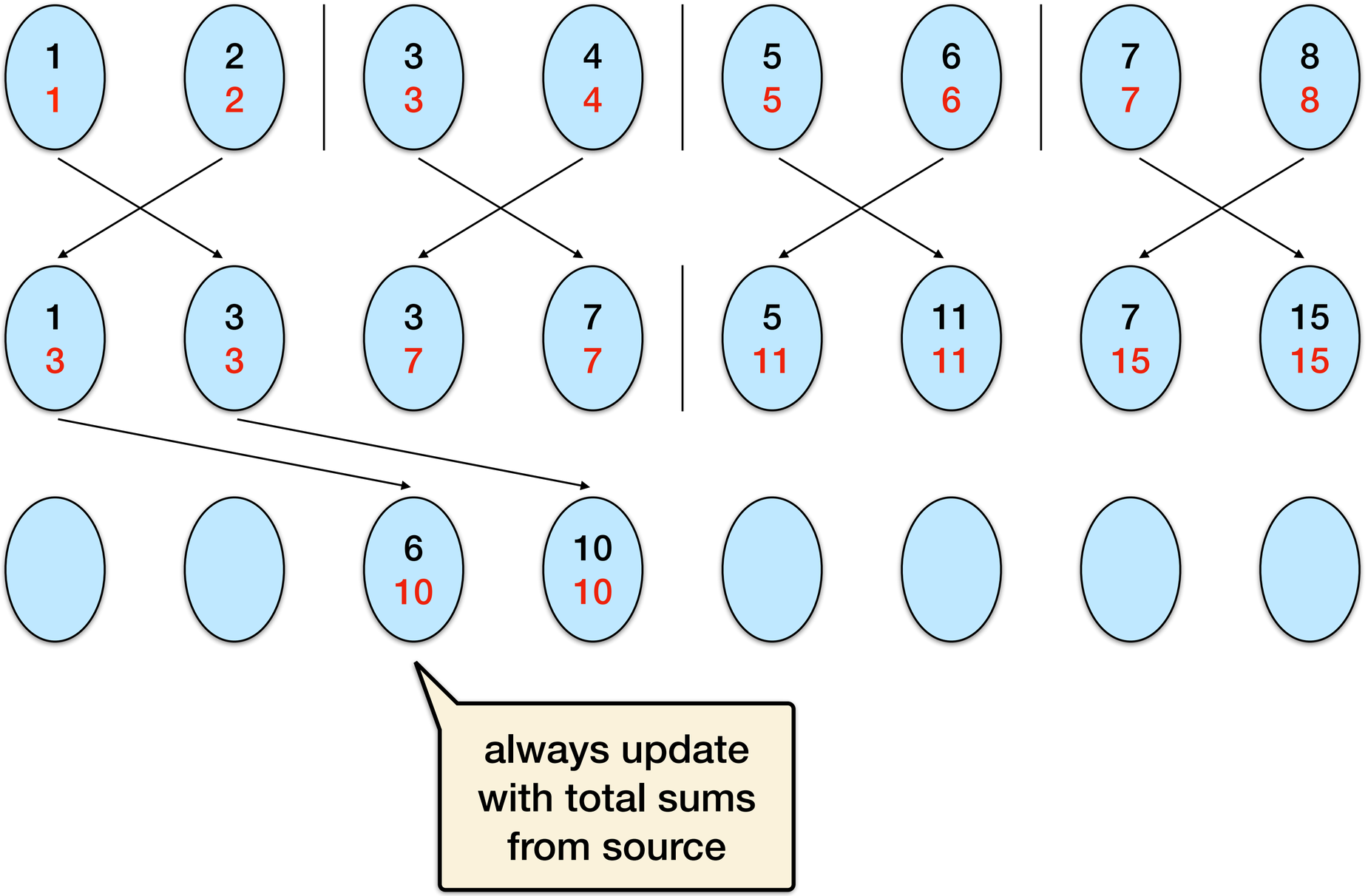

1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
Initialize prefix and total sum as the input number

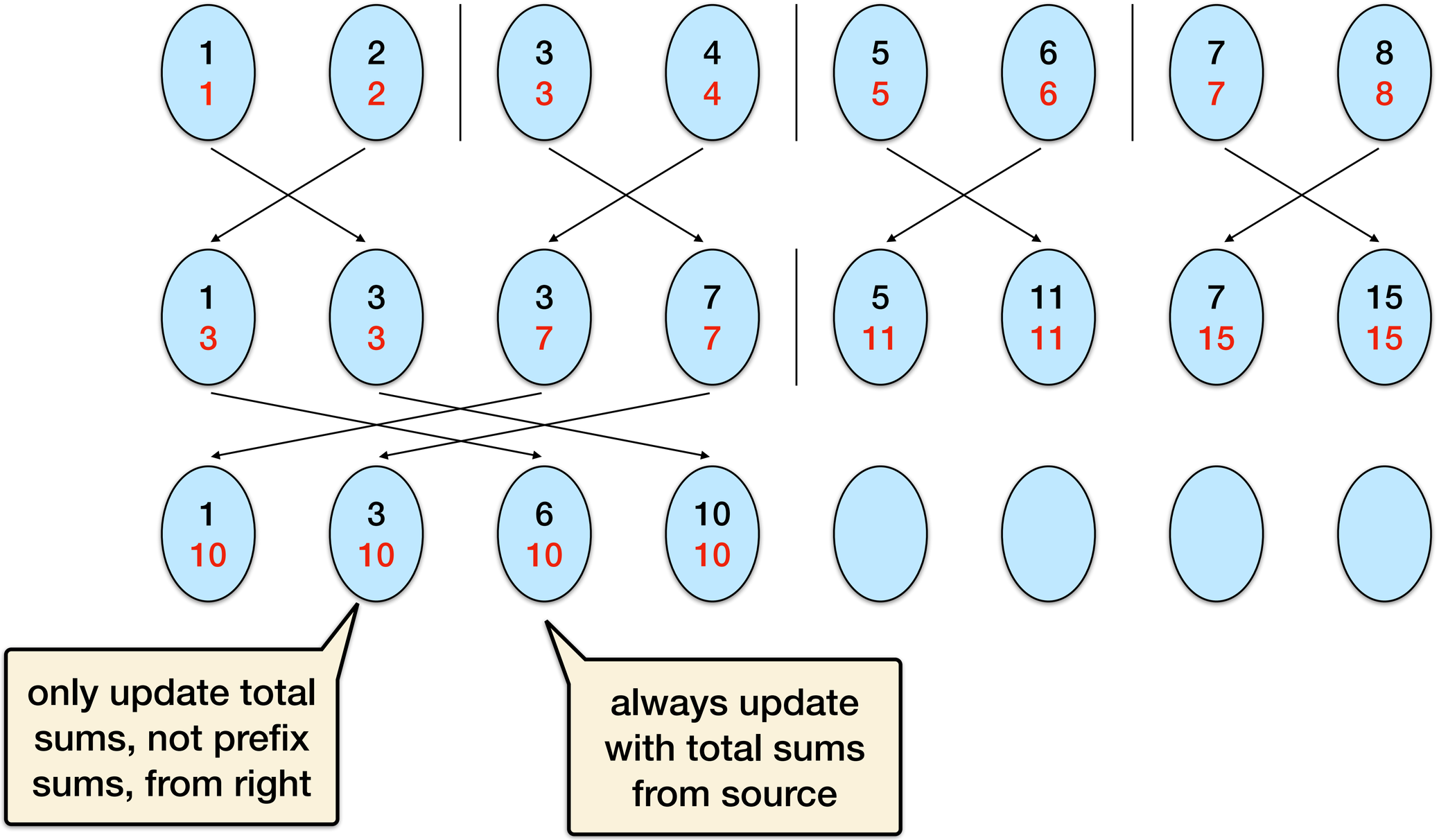

1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
3
3
If from the left, add total sum to prefix and total

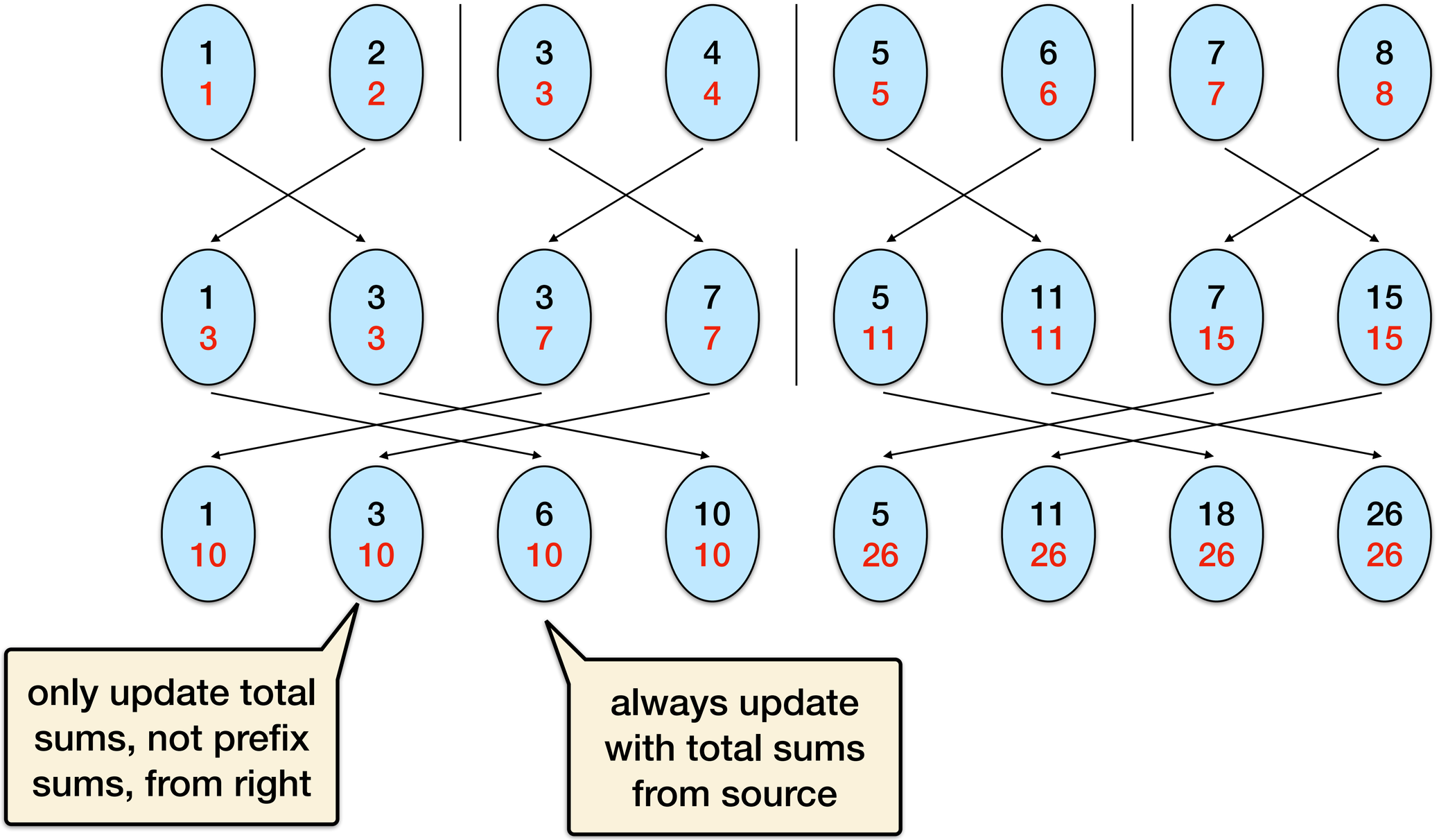

1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
1
3
3
3
If from the right, add total sum to total sum
If from the left, add total sum to prefix and total

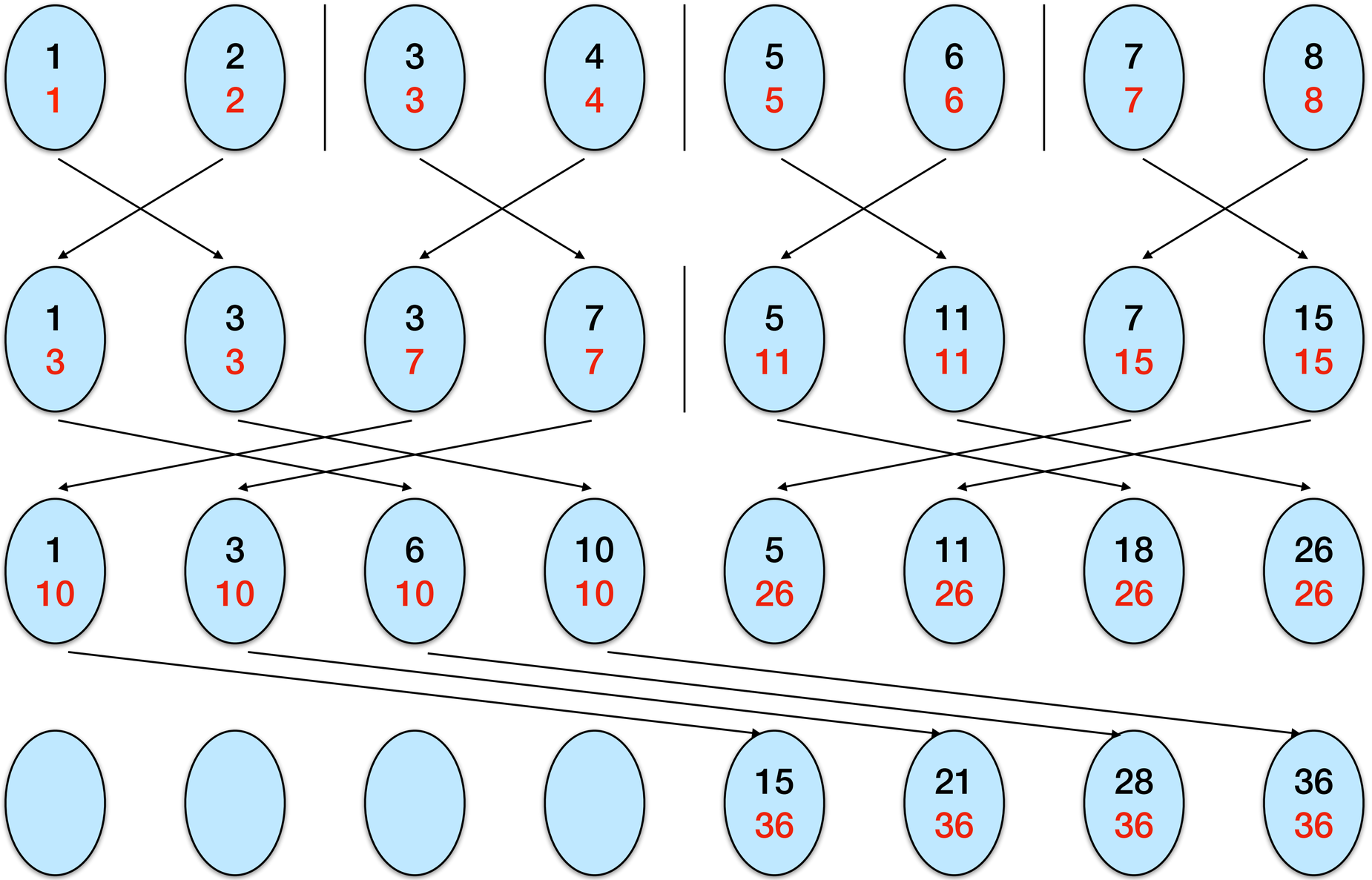

1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
1
3
3
3
3
7
7
7
If from the right, add total sum to total sum
If from the left, add total sum to prefix and total

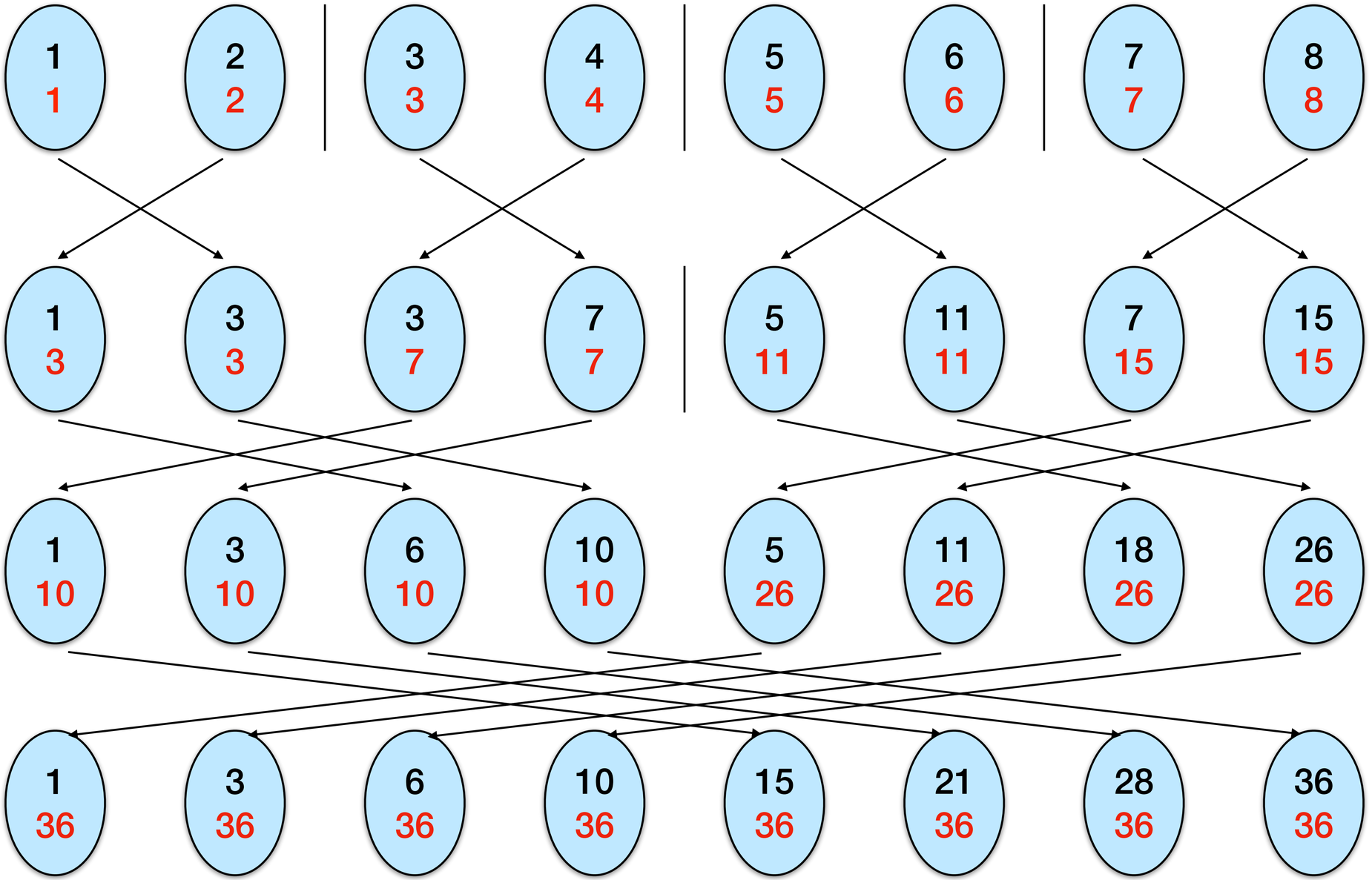

1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
1
3
3
3
7
7
5
11
11
11
7
15
15
15
If from the right, add total sum to total sum
If from the left, add total sum to prefix and total


1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
1
3
3
3
7
7
5
11
11
11
7
15
15
15
6
10
10
10
always update with total sums from source


1
1
2
2
3
3
4
4
5
5
6
6
7
7
8
8
1
3
3
3
7
7
5
11
11
11
7
15
15
15
1
10
3
10
6
10
10
10
only update total sums, not prefix sums, from right
always update with total sums from source


1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
1 3
3 3
3 7
7 7
5 11
11 11
7 15
15 15
1 10
3 10
6 10
10 10
5 26
11 26
18 26
26 26
only update total sums, not prefix sums, from right
always update with total sums from source


1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
1 3
3 3
3 7
7 7
5 11
11 11
7 15
15 15
1 10
3 10
6 10
10 10
5 26
11 26
18 26
26 26
15 36
21 36
28 36
36 36


1 2 3 4 5 6 7 8
1 2 3 4 5 6 7 8
1 3 3 7 5 11 7 15
3 3 7 7 11 11 15 15
1 3 6 10 5 11 18 26
10 10 10 10 26 26 26 26
1 3 6 10 15 21 28 36
36 36 36 36 36 36 36 36

Pseudocode for Parallel Prefix

```
Algorithm (for P_i)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Pseudocode for Parallel Prefix

```
Algorithm (for P_i)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Initialize prefix and total sum as the input number

From Srinivas Aluru

Pseudocode for Parallel Prefix

```
Algorithm (for Pi)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Initialize prefix and total sum as the input number

Find neighbor in this round

Pseudocode for Parallel Prefix

```
Algorithm (for Pi)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Initialize prefix and total sum as the input number

Find neighbor in this round

Send/receive total sums

Pseudocode for Parallel Prefix

```
Algorithm (for Pi)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Initialize prefix and total sum as the input number

Find neighbor in this round

Send/receive total sums

Always update total sum

Pseudocode for Parallel Prefix

```
Algorithm (for P_i)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Initialize prefix and total sum as the input number

Find neighbor in this round

Send/receive total sums

Always update total sum

Update prefix sum if sum is coming from the left

Pseudocode for Parallel Prefix

```
Algorithm (for P_i)
total_sum ← prefix_sum ← local_number
for j=0 to d-1 do
    rank' ← rank XOR 2^j
    send total_sum to rank'
    receive received_sum from rank'
    total_sum ← total_sum + received_sum
    if (rank > rank')
        prefix_sum ← prefix_sum + received_sum
endfor
```

Initialize prefix and total sum as the input number

Find neighbor in this round

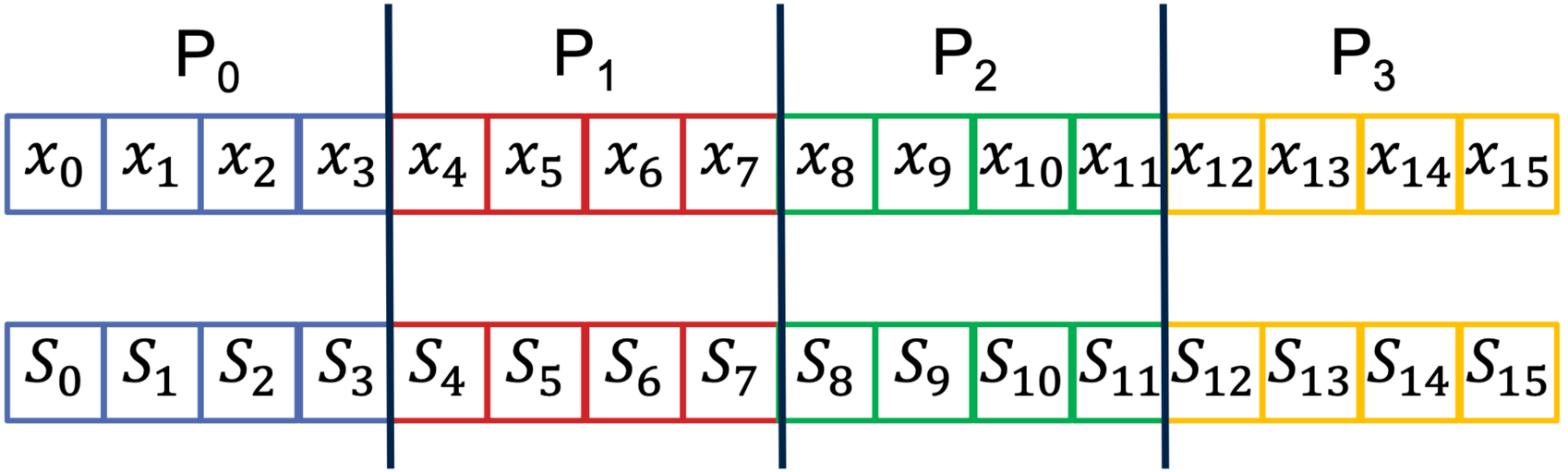
Send/receive total sums

Always update total sum

Update prefix sum if sum is coming from the left

Next: what if n > p?

Parallel Prefix with Fewer Processors



- Use Brent's Lemma?
- $T(n,p) = \Theta\left(\frac{n}{p}\log n\right)$

Algorithm Summary

1. Compute prefix sum locally on each processor

2. Perform parallel prefix sum using the last local prefix sum on each processor

3. Add the result of parallel prefix sum on the processor to each of its local prefix sums

Computation time = $\Theta(\frac{n}{p} + \lg p)$

Communication time = $\Theta((\tau + \mu)\lg p)$

Bandwidth in s/byte for convenience

From Srinivas Aluru

Algorithm Summary

1. Compute prefix sum locally on each processor
2. Perform parallel prefix sum using the last local prefix sum on each processor
3. Add the result of parallel prefix sum on the processor to each of its local prefix sums

Is this correct?

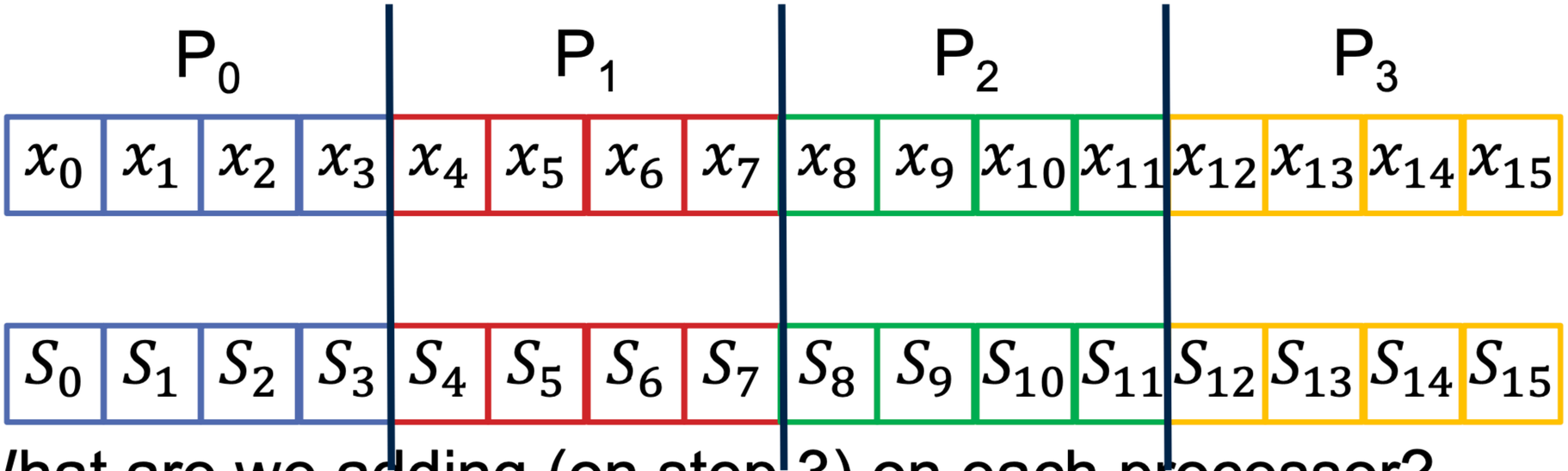
Computation time = $\Theta(\frac{n}{p} + \lg p)$

Communication time = $\Theta((\tau + \mu)\lg p)$

Bandwidth in s/byte for convenience

From Srinivas Aluru

Algorithm Example



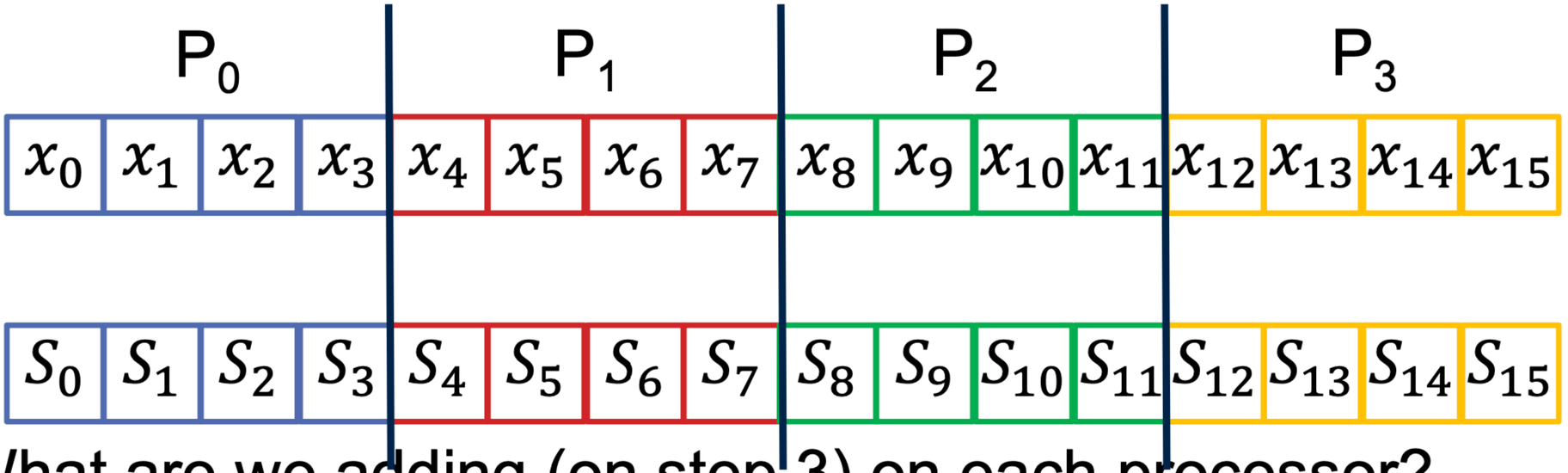
- What are we adding (on step 3) on each processor?
  - P_0 adds S_3
  - P_1 adds S_7
  - P_2 adds S_{11}
  - P_3 adds S_{15}

Should not be adding end to self

Ideas about how to fix it?

From Srinivas Aluru

Algorithm Example



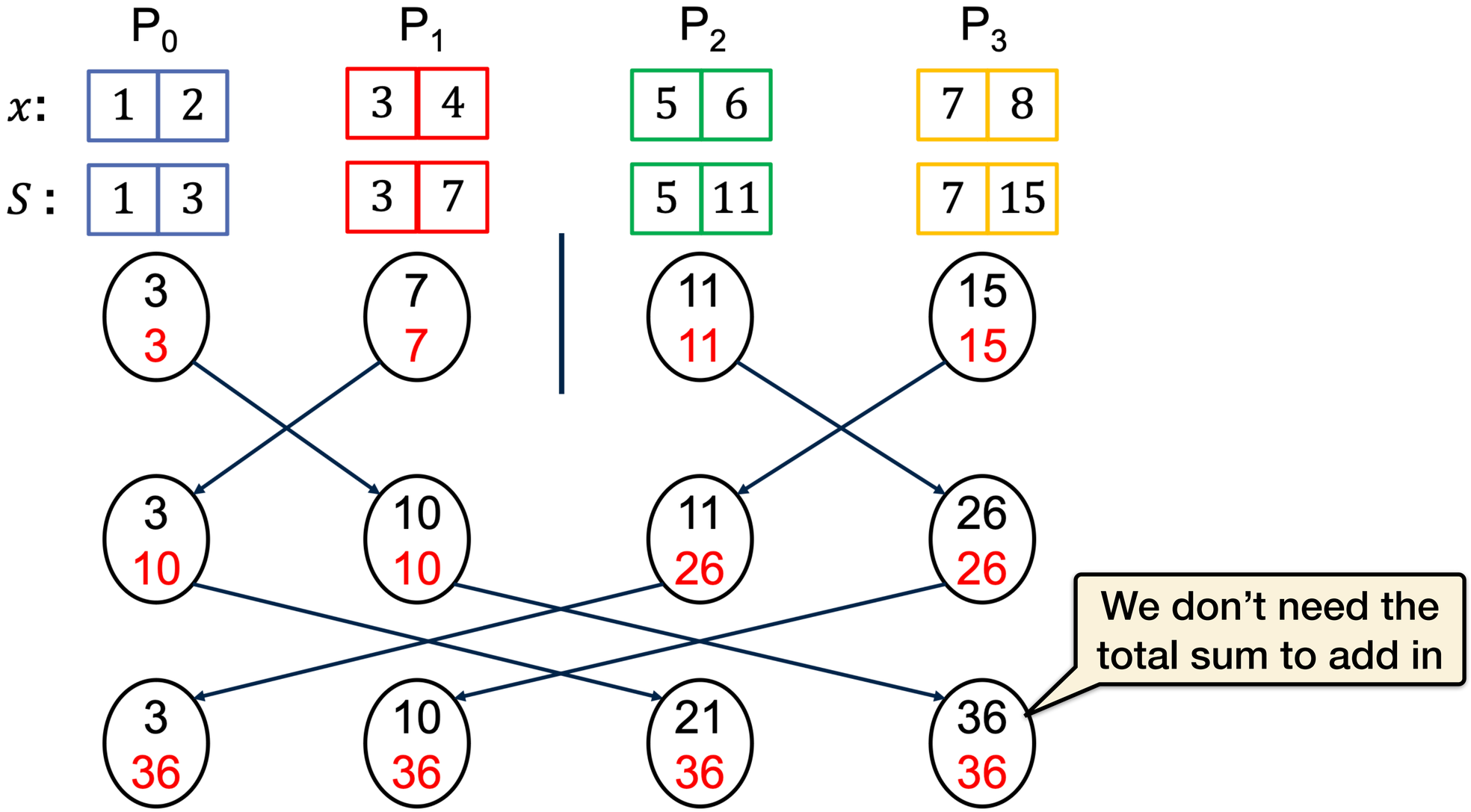
- What are we adding (on step 3) on each processor?
  - P_0 adds S_3
  - P_1 adds S_7
  - P_2 adds S_{11}
  - P_3 adds S_{15}

Should not be adding end to self

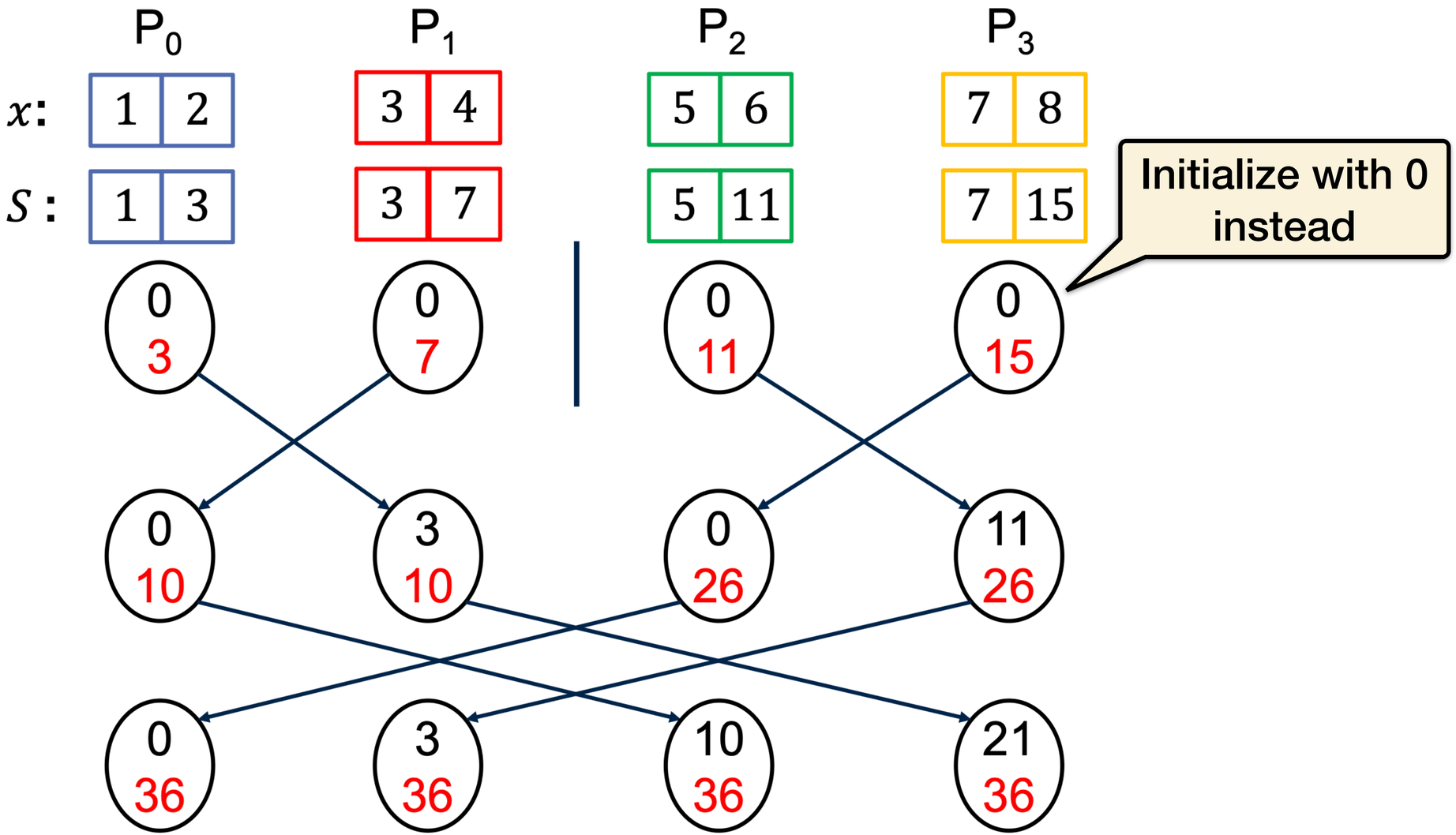
Potential solution: Exclusive scan

From Srinivas Aluru

With Outer Inclusive Scan



With Outer Exclusive Scan



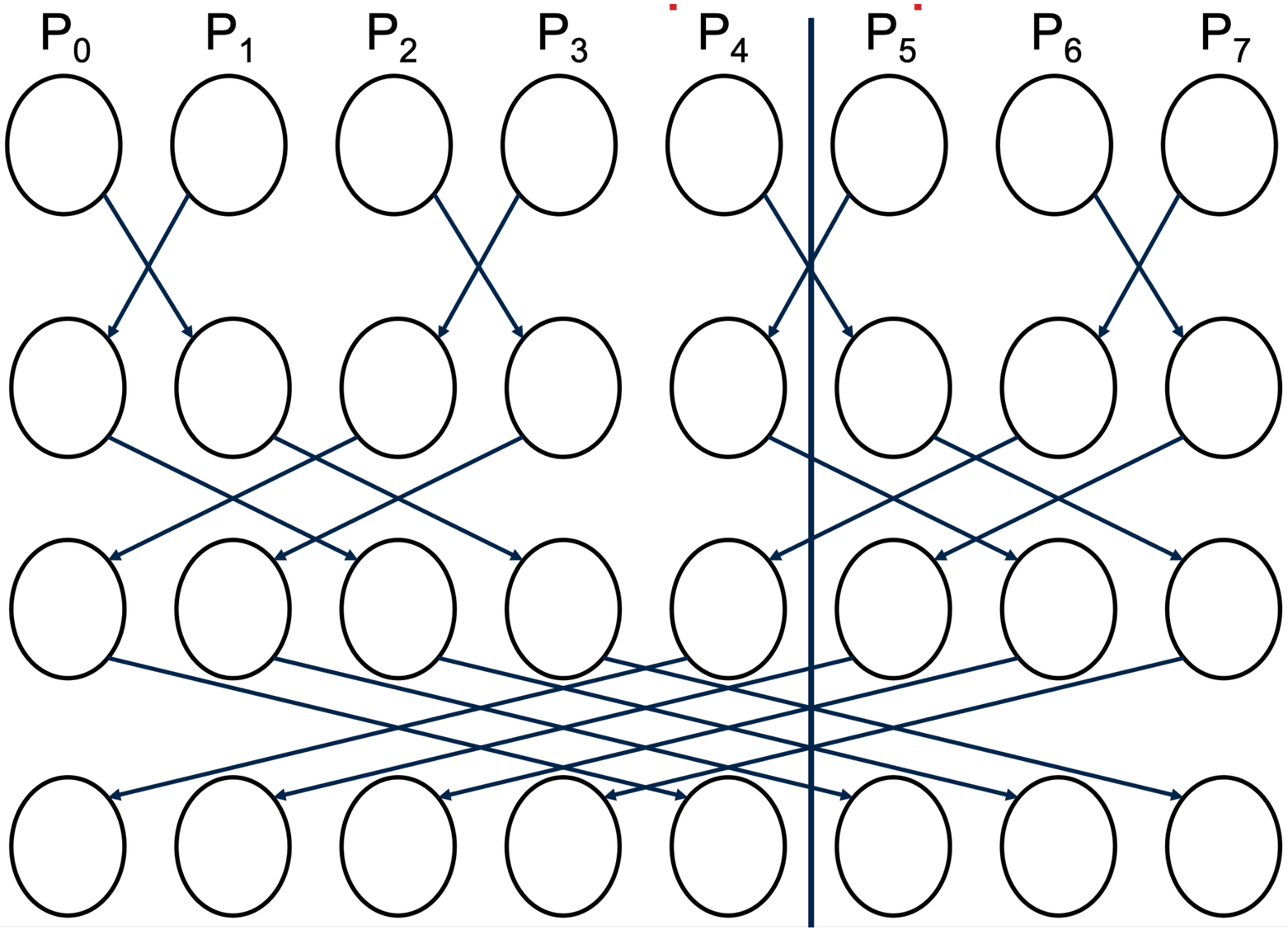
Generalizing Parallel Prefix

If n is not divisible by p, assign some processors with 1 more element than the others.

What if p is not a power of 2?

- Find p' = a power of 2 such that $\frac{p'}{2} < p < p'$.
- Run your code like you have p' processors.
- Ignore communications to/from non-existing processors (i.e., rank $\geq p$)

Parallel Prefix on Non-Power-of-2 Processors



From Srinivas Aluru